Microservices Strategy Debate

Interviewer:

Our monolith is becoming slow and hard to scale.

The Tech lead suggested: Break every single feature into its own microservice.

What is your reply?

I would **push back a bit** on that suggestion — not reject it completely, but refine it.

💡 My reply in an interview

Breaking *every single feature* into its own microservice is usually a **bad idea**.

Microservices are not about *maximum splitting*, they are about **right splitting**.

🚫 Why this approach is risky

If we blindly convert every feature into a microservice:

1. **Too many services (micro → nano services 😅)**
   - Hard to manage
   - Deployment becomes complex
   - Monitoring/debugging nightmare
2. **Network overhead increases**
   - Earlier: method calls (fast)
   - Now: HTTP/gRPC calls (slow, failure-prone)
3. **Data consistency problems**
   - Transactions across services become difficult
   - Need patterns like Saga
4. **Operational complexity**
   - Requires:
     - Service discovery
     - Load balancing
     - Distributed logging
     - Fault tolerance

✅ Better approach (what I would suggest)

Instead of splitting by *features*, split by **business domains (bounded contexts)**

👉 Follow **Domain-Driven Design (DDD)**

🔹 Step-by-step strategy

1. **Identify bounded contexts**
   Example:
   - User Service
   - Order Service
   - Payment Service
   - Inventory Service
2. **Start with modular monolith**
   - Keep code modular inside same app
   - Reduce coupling first
3. **Extract high-impact services**
   - Services that:
     - Scale differently (e.g., payments, search)
     - Change frequently
     - Have clear boundaries
4. **Use microservices where justified**
   Not everywhere.

🔹 When microservices make sense

- Independent scaling needed
- Different teams own different domains
- High traffic on specific modules
- Technology flexibility required

🔹 When NOT to use them

- Small team
- Early-stage product
- Tight coupling between features
- No DevOps maturity

🎯 Final interview punchline

> "I wouldn't break every feature into a microservice. That leads to unnecessary complexity. Instead, I'd identify proper domain boundaries using DDD, start with a modular monolith, and gradually extract services where there is a clear need for independent scaling or ownership."

If you want, I can also give you:

- A **real-world example (like Netflix or Amazon)**
- Or a **diagram showing bad vs good microservice split**

> Instead, I'd identify proper domain boundaries using DDD, start with a modular monolith, and gradually extract services where there is a clear need for independent scaling or ownership."

That's actually a **very strong answer already** 👍 — but in an interview, you can make it even sharper by explaining *how* you would do it.

Let me expand your statement so you can confidently defend it if the interviewer asks "how exactly?" 👇

🔹 What you said (core idea)

> "Identify proper domain boundaries using Domain-Driven Design (DDD), start with a modular monolith, and gradually extract services..."

✔️ This shows:

- You understand **architecture trade-offs**
- You are **not blindly following trends**

🔹 How you would actually do it (important!)

1. Identify Bounded Contexts (DDD)

Break system by **business capability**, not by technical layers.

Example:

Wrong Split ❌	Correct Split ✅
Auth Service	User Service
DB Service	Order Service
API Service	Payment Service

👉 Each service should:

- Have its own **business logic**

- Own its **data**
- Be **loosely coupled**

---

2. Start with Modular Monolith

Before microservices, structure code like:

```
com.app
 ├── user
 ├── order
 ├── payment
 ├── inventory
```

- Clear separation
- Minimal cross-module dependency
- Internal APIs between modules

👉 This reduces risk before going distributed

---

3. Identify Extraction Candidates

Not everything should become a microservice.

Extract only when:

- 🔥 High traffic (e.g., payments, search)
- 🔄 Frequent changes (independent deployments needed)
- 👥 Separate team ownership
- ⚖️ Different scaling needs

---

4. Extract Gradually

Example flow:

`Monolith → Modular Monolith → Extract Order Service → Extract Payment Service`

- Use API calls between services
- Keep system stable while migrating

---

5. Handle Distributed Challenges

Once you extract:

- Use **event-driven communication** (Kafka/RabbitMQ)
- Apply patterns like Saga for transactions
- Add observability (logs, tracing)

🔹 Why this approach is better

- Avoids **over-engineering**
- Reduces **failure points**
- Keeps system **maintainable**
- Allows **incremental scaling**

🎯 Final polished interview answer

You can say it like this:

> "I would avoid splitting every feature into a microservice because that introduces unnecessary complexity. Instead, I'd use Domain-Driven Design to identify bounded contexts and first structure the system as a modular monolith. Then, based on scaling needs, team ownership, and change frequency, I'd gradually extract specific domains into microservices. This ensures we get the benefits of microservices without the operational overhead too early."

If you want, I can simulate a **follow-up grilling round** (like interviewer asking tricky questions on this).

Data consistency problems

Transactions across services become difficult
Need patterns like Saga transaction means

Good question — this is exactly where many candidates get stuck in interviews.

🔹 What is a "transaction"?

In simple terms:

> A **transaction** is a group of operations that must behave as **one single unit** — either everything succeeds or everything fails.

This concept comes from databases (ACID properties).

✅ Example (Monolith case)

Let's say user places an order:

1. Create Order
2. Deduct Balance

3. Reduce Inventory

👉 In a monolith + single DB:

- All steps run inside **one DB transaction**
- If step 2 fails → everything is rolled back

✔️ This is called **atomicity**

🔹 Problem in Microservices

Now imagine same flow but split:

- Order Service → creates order
- Payment Service → deducts money
- Inventory Service → updates stock

👉 Each service has **its own database**

❌ You CANNOT have one single DB transaction across all services

---

🚨 What can go wrong?

- Order created ✅
- Payment failed ❌
- Inventory reduced ❌ (already done)

👉 System becomes **inconsistent**

🔹 So what do we do?

We use patterns like:

👉 Saga Pattern

🔹 What is Saga Pattern?

Instead of one big transaction:

👉 Break into **multiple local transactions**

Each step:

- Executes independently
- Has a **compensating action (undo)**

🔁 Example with Saga

1. Order Created ✅
2. Payment Deducted ❌

👉 Then trigger compensation:

- Cancel Order ❌ (undo step 1)

🔄 Flow looks like:

```
Order → Payment → Inventory
   ↓        ❌
Cancel Order (compensation)
```

🔹 Two types of Saga

1. **Choreography**
   - Services communicate via events
   - No central controller
2. **Orchestration**
   - One service controls the flow (orchestrator)

🔹 Key Interview Line

> "In microservices, we can't use traditional ACID transactions across services, so we use patterns like Saga where each service performs a local transaction and defines compensating actions to maintain eventual consistency."

🔹 One-liner to remember

👉 **Monolith → Strong consistency (ACID)**
👉 **Microservices → Eventual consistency (Saga)**

If you want, I can also:

- Show **Spring Boot Saga implementation**
- Or explain **2PC vs Saga (very common interview trap)**

---

can we use a combination of micro and monolith

Yes — and not only *can* you, **you should in many cases** 👍

This approach is actually very common and practical.

🔹 What is it called?

It's often referred to as a:

👉 **Hybrid Architecture** (Monolith + Microservices)

or

👉 **Modular Monolith + Microservices**

🔹 Core Idea

> Keep most of the system as a **monolith**, and extract only **specific parts** as microservices.

🔹 Example

Let's say you have an e-commerce app:

🏢 Monolith (main app)

- User management
- Product catalog
- Order management

⚡ Microservices (separate)

- Payment Service (high traffic, critical)
- Notification Service (email/SMS async)
- Search Service (needs scaling, maybe Elasticsearch)

🔹 Why this works well

✅ 1. Controlled complexity

- You don't introduce distributed system problems everywhere

✅ 2. Better performance

- Internal monolith calls are faster than network calls

✅ 3. Gradual migration

- You can slowly move toward microservices

✅ 4. Team scalability

- Teams can own specific services independently

🔹 When to extract a service

Only move something out of monolith if:

- 🚀 Needs independent scaling
- 🔄 Changes frequently
- ⚙️ Requires different tech stack
- 🔗 Has clear boundaries

🔹 Real-world approach

Many companies started like this:

- Start with monolith
- Make it modular
- Extract critical services over time

🔹 Architecture Flow

```
Monolith → Modular Monolith → Hybrid → Microservices (if needed)
```

🔹 Important caution ⚠️

Hybrid architecture can become messy if:

- Boundaries are unclear
- Too many dependencies between monolith & services
- Shared database is used incorrectly

👉 Always maintain:

- Clear APIs
- Independent data ownership

🎯 Interview-ready answer

> “Yes, a hybrid approach is often the most practical. I’d keep a modular monolith for most features and extract only those components into microservices that need independent scaling, frequent changes, or separate ownership. This helps balance performance, simplicity, and scalability without introducing unnecessary distributed system complexity.”

If you want, I can also explain:

- 🔥 “Strangler Fig Pattern” (used to slowly break monolith)
- 🔥 Real example (Netflix, Amazon migration journey)
- 🔥 Diagram showing hybrid architecture visually